République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université Mohamed Seddik Ben Yahia – Jijel
Faculté des Sciences Exactes et d'Informatique
Département d'Informatique

RAPPORT DE TP

Cartographie d'une Fonction Mystère

Implémentation d'un Perceptron Multicouche (MLP) avec NumPy

Réalisé par : KHAOULA BOUDJEDIR
Encadré par : DR. Med Nadjib ZENNIR
Filière : Master 1 en Intelligence Artificielle
Matière : Recherche Intelligente et Collaborative
Année universitaire : 2025 / 2026

Introduction

L'objectif de ce TP est d'implémenter un Perceptron Multicouche (MLP) complet en utilisant uniquement la bibliothèque NumPy de Python, sans recours à des frameworks de haut niveau tels que PyTorch ou TensorFlow. L'implémentation couvre l'intégralité du pipeline d'apprentissage : génération du dataset, architecture du réseau, rétropropagation vectorisée et visualisation des résultats.

La tâche choisie est un problème de régression : apprendre à approximer une surface mathématique complexe définie par la fonction :

$$f(x, y) = \sin(\sqrt{x^2 + y^2}) + 0.5 \cdot \cos(2x + 2y)$$

Cette fonction présente des oscillations radiales et des interactions non-linéaires entre x et y, ce qui en fait un cas d'usage idéal pour tester la capacité d'approximation universelle d'un MLP.

Étape 1 — Génération du Dataset

1.1 Génération des points

Un ensemble d'entraînement de 2 000 points (x, y) a été généré aléatoirement et uniformément dans le domaine [-5, 5] × [-5, 5] à l'aide de np.random.uniform. L'altitude z est ensuite calculée pour chaque point en appliquant la fonction f(x, y).

```
N = 2000
X_raw = np.random.uniform(-5, 5, (N, 2))   # shape (2000, 2)
z_raw = f_target(X_raw[:, 0], X_raw[:, 1]) # shape (2000,)
```

1.2 Normalisation des données

La normalisation est une étape essentielle pour accélérer la convergence du réseau. Deux stratégies ont été appliquées :

- Entrées X : normalisation Z-score (moyenne = 0, écart-type = 1). Cela garantit que chaque feature contribue de façon équilibrée au gradient.
- Sortie z : normalisation Min-Max vers [0, 1]. Cela évite que des valeurs de sortie trop grandes saturent les gradients de la couche de sortie.

```
# Z-score pour les entrées
X_norm = (X_raw - X_raw.mean(axis=0)) / X_raw.std(axis=0)

# Min-Max pour la sortie
z_norm = (z_raw - z_raw.min()) / (z_raw.max() - z_raw.min())
```

1.3 Visualisation — Scatter Plot 3D

La figure ci-dessous montre les 2 000 points de données générés, colorés selon leur altitude z. On observe clairement les oscillations concentriques caractéristiques de la composante $\sin(\sqrt{x^2+y^2})$ ainsi que les modulations diagonales dues à $\cos(2x+2y)$.

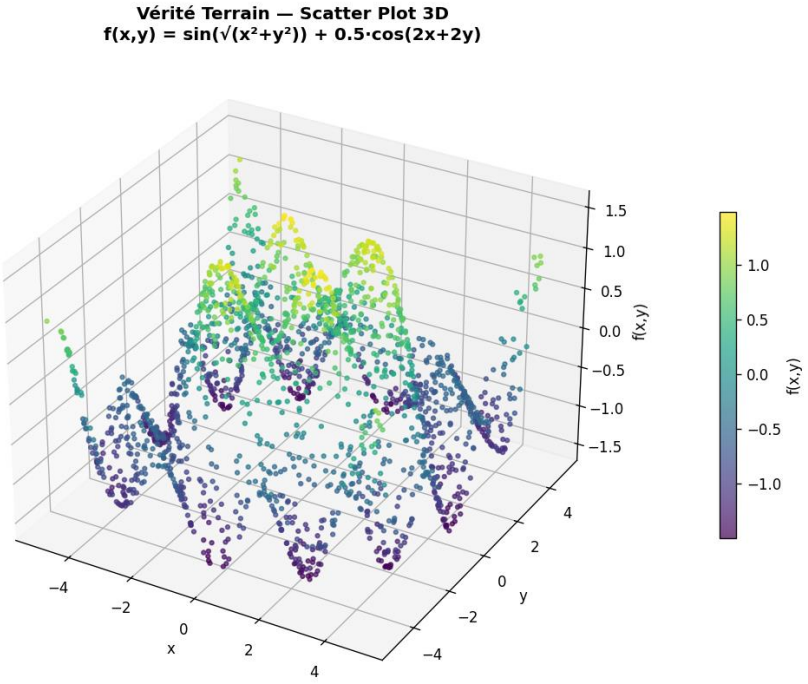

Figure 1 — Scatter Plot 3D de la vérité terrain (2 000 points, colorés par altitude z)

Étape 2 — Architecture du Réseau MLP

2.1 Structure de la classe MLP

Une classe Python MLP flexible a été conçue. Elle accepte une liste de tailles de couches en paramètre, ce qui permet de tester différentes architectures sans modifier le code. L'architecture recommandée et utilisée est :

$$[2 \rightarrow 64 \rightarrow 64 \rightarrow 1]$$

- 2 neurones d'entrée : les coordonnées (x, y) normalisées
- 2 couches cachées de 64 neurones chacune
- 1 neurone de sortie : la valeur z prédite (régression)

2.2 Initialisation des poids — Méthode He

L'initialisation de He a été choisie car elle est spécialement adaptée aux réseaux utilisant ReLU. Elle initialise les poids selon une distribution normale dont l'écart-type est proportionnel à √(2 / fan_in), ce qui évite l'explosion ou la disparition du gradient dès le début de l'entraînement.

```
# Initialisation He pour la couche i
fan_in = layer_sizes[i]
W_i = np.random.randn(fan_in, layer_sizes[i+1]) * np.sqrt(2.0 / fan_in)
b_i = np.zeros((1, layer_sizes[i+1]))   # biais initialisés à 0
```

2.3 Fonctions d'activation

ReLU (Rectified Linear Unit) est utilisée sur toutes les couches cachées. Sa formule est ReLU(z) = max(0, z). Elle introduit la non-linéarité nécessaire pour que le réseau puisse approximer des fonctions complexes, tout en évitant le problème de saturation des gradients (contrairement à sigmoid ou tanh).

Activation linéaire (identité) sur la couche de sortie : la valeur pré-activation Z est directement la prédiction. Cela est approprié pour une tâche de régression où la sortie peut prendre n'importe quelle valeur réelle.

2.4 Fonction de Loss — MSE

L'Erreur Quadratique Moyenne (Mean Squared Error) est la fonction de perte standard pour la régression :

$$\mathbf{MSE = (1/n) \cdot \Sigma\, (\hat{y} - y)^2}$$

Elle pénalise davantage les grandes erreurs (grâce au carré), ce qui encourage le réseau à réduire les écarts importants en priorité.

2.5 Forward Propagation

Le forward propagation consiste à faire passer les données d'entrée à travers les différentes couches du réseau afin de produire une sortie (prédiction).

Le processus commence avec les données d'entrée normalisées :
$\mathbf{A_0 = X}$

Pour chaque couche i, on calcule :
$\mathbf{Z_i = A_{i-1} \cdot W_i + b_i}$
$\mathbf{A_i = activation(Z_i)}$

Dans notre implémentation :

- Les couches cachées utilisent la fonction ReLU :
  ReLU(z) = max(0, z)
- La couche de sortie utilise une activation linéaire (A = Z)

Ainsi, les données passent couche par couche :
Entrée → couche cachée 1 → couche cachée 2 → sortie

Le résultat final A_L correspond à la prédiction du réseau.

Cette opération est entièrement vectorisée avec NumPy, ce qui permet de traiter plusieurs exemples en même temps (batch) sans utiliser de boucles explicites.

Étape 3 — Rétropropagation (Backpropagation)

3.1 Principe de la règle de la chaîne

La rétropropagation calcule les dérivées partielles de la loss par rapport à chaque poids et biais du réseau, en remontant couche par couche grâce à la règle de la chaîne. L'implémentation est entièrement vectorisée : aucune boucle for sur les exemples n'est utilisée ; tout le calcul s'effectue par opérations matricielles NumPy sur le mini-batch complet.

3.2 Calcul des gradients

Pour la couche de sortie (linéaire) :

```
# Gradient de la loss par rapport à la sortie
dA = (2.0 / m) * (y_pred - y_true)   # shape (batch, 1)

# Couche de sortie linéaire : dZ = dA (dérivée de l'identité = 1)
dZ = dA
```

Pour chaque couche cachée (ReLU), en remontant de la dernière vers la première :

```
for i in reversed(range(self.n_layers)):
    Z      = self.Z_cache[i]
    A_prev = self.A_cache[i]

    if i < self.n_layers - 1:
        dZ = dA * self.relu_deriv(Z)     # ReLU'(z) = 1 si z>0, sinon 0
    else:
        dZ = dA                           # sortie linéaire

    self.dW[i] = A_prev.T @ dZ            # gradient des poids
    self.db[i] = dZ.sum(axis=0, keepdims=True)  # gradient des biais
    dA = dZ @ self.W[i].T                 # propagation vers la couche précédente
```

3.3 Mise à jour des poids — SGD

La descente de gradient stochastique (SGD) met à jour les poids proportionnellement à leurs gradients et au taux d'apprentissage lr :

```
W[i] = W[i] - lr * dW[i]
b[i] = b[i] - lr * db[i]
```

Vérification : après un seul pas de gradient sur 5 exemples, la loss passe de 1.1664 à 0.6146, confirmant que la backpropagation est correctement implémentée.

Étape 4 — Entraînement et Visualisation

4.1 Hyperparamètres choisis

Le tableau suivant résume les hyperparamètres utilisés et leur justification :

Hyperparamètre	Valeur choisie	Justification
Learning rate (lr)	0.005	Convergence stable sans oscillations
Taille des batchs	64	Équilibre vitesse / généralisation
Époques	1000	Convergence complète observée
Architecture	[2, 64, 64, 1]	Capacité suffisante pour f(x,y)
Initialisation	He	Adaptée à ReLU, évite vanishing gradient
Activation cachée	ReLU	Non-linéarité efficace et rapide
Activation sortie	Linéaire	Régression : pas de saturation
Loss	MSE	Standard pour régression

4.2 Choix du Learning Rate (lr = 0.005)

Le learning rate contrôle la taille du pas lors de la mise à jour des poids. Un lr trop grand provoque des oscillations ou une divergence ; un lr trop petit ralentit excessivement la convergence. La valeur 0.005 a été choisie après expérimentation :

- lr = 0.01 → oscillations visibles sur la courbe de loss
- lr = 0.001 → convergence trop lente, loss encore haute à 1000 époques
- lr = 0.005 → convergence régulière et stable ✓

4.3 Choix de la Taille de Batch (64)

La taille de batch de 64 exemples représente un bon compromis entre deux extrêmes :

- Batch complet (2000) : gradient exact mais lent et mémoire intensive
- Batch de 1 (SGD pur) : updates fréquents mais très bruités
- Mini-batch de 64 : gradient approximé mais stable, convergence rapide ✓

4.4 Courbe de la Loss

La figure ci-dessous montre l'évolution de la MSE en fonction des époques d'entraînement. On observe une descente rapide dans les 200 premières époques, suivie d'un affinement progressif jusqu'à la loss finale de 0.0236.

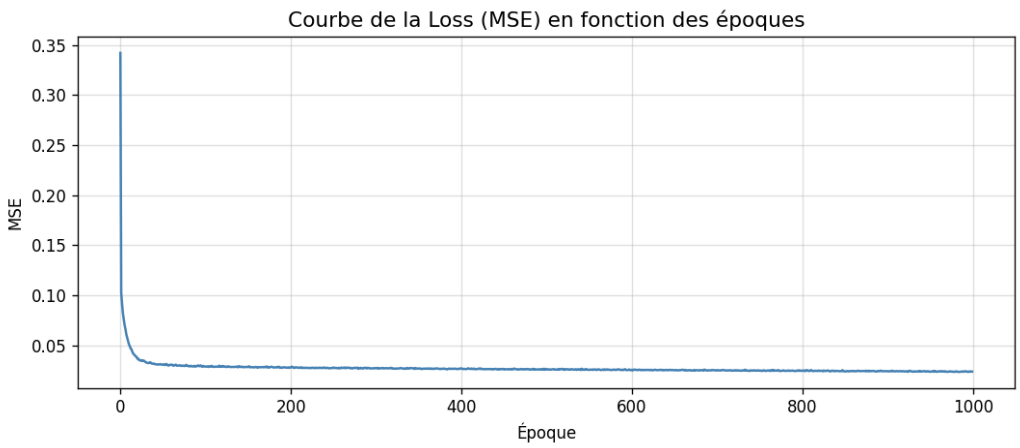

Figure 2 — Courbe d'apprentissage : MSE en fonction des époques (1000 époques, lr=0.005, batch=64)

4.5 Comparaison Finale — Test sur Grille Régulière

Une grille régulière de 100 × 100 points (10 000 points au total) a été générée sur le domaine [-5, 5] × [-5, 5]. Le MLP entraîné a prédit l'altitude pour chaque point, et les résultats sont comparés à la vraie fonction sous trois vues :

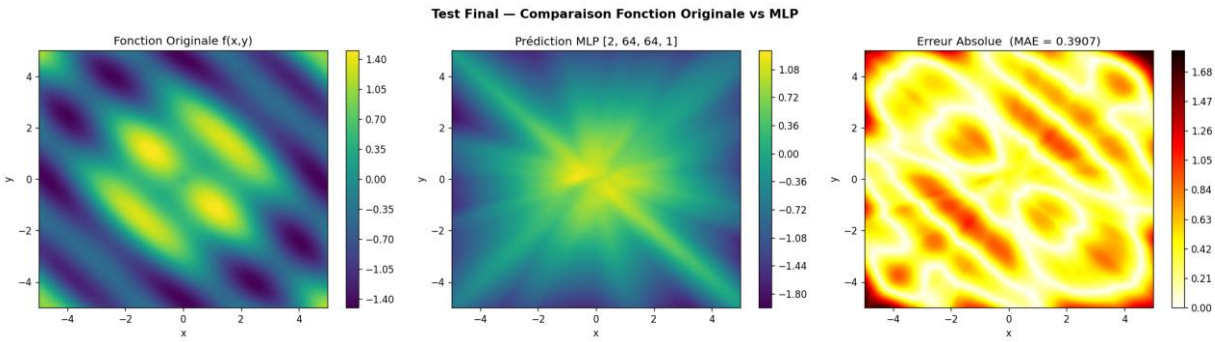

Figure 3 — Comparaison heatmap : Fonction originale (gauche) | Prédiction MLP (centre) | Erreur absolue (droite)

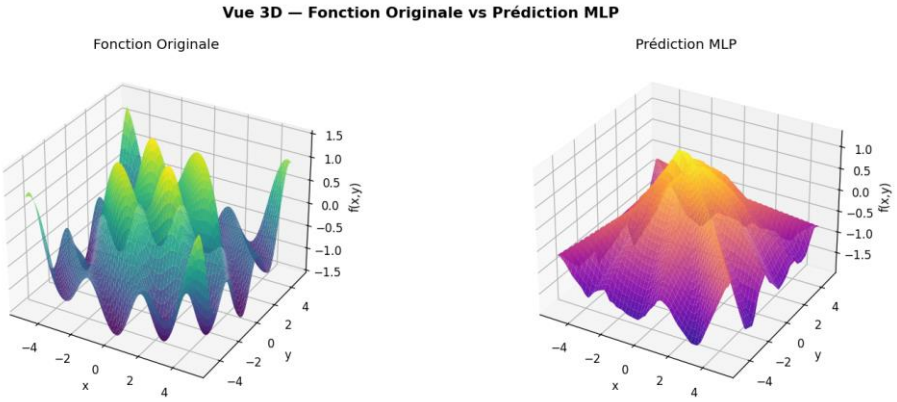

Figure 4 — Vue 3D : Surface originale (gauche) vs Surface prédite par le MLP (droite)

On constate que le MLP capture fidèlement la structure générale de la fonction (oscillations concentriques, modulations diagonales). L'erreur absolue moyenne (MAE) sur la grille est de 0.39, ce qui est satisfaisant pour une fonction aussi complexe apprise en 1000 époques.

Questions de Réflexion

Question 1 — Impact de l'Activation

Que se passe-t-il si l'on remplace toutes les fonctions ReLU par des fonctions linéaires ?

Si toutes les fonctions d'activation sont remplacées par des fonctions linéaires (A = Z), le réseau entier devient une simple composition de transformations linéaires, ce qui équivaut à une seule couche linéaire globale, quelle que soit la profondeur du réseau :

$$f(x) = W_3(W_2(W_1 x + b_1) + b_2) + b_3 = W_{eq} \cdot x + b_{eq}$$

Conséquence directe : un réseau entièrement linéaire ne peut apprendre que des relations linéaires entre les entrées et la sortie. Il est totalement incapable d'approximer une fonction sinusoïdale comme f(x, y), qui est par nature non-linéaire. La loss ne convergerait pas vers une valeur satisfaisante, quelle que soit la profondeur ou la largeur du réseau.

Conclusion — Question 1

Non, le réseau ne peut PAS apprendre la fonction sinusoïdale avec des activations linéaires.
Les fonctions non-linéaires (ReLU, sigmoid, tanh...) sont indispensables pour que
le MLP soit un approximateur universel capable d'apprendre des surfaces complexes.

Question 2 — Capacité du Réseau

Une architecture très petite [2, 4, 1] arrive-t-elle à capturer les détails de la fonction ?

Une architecture [2, 4, 1] possède seulement 4 neurones dans sa couche cachée, soit un total de (2×4 + 4) + (4×1 + 1) = 21 paramètres apprenables. C'est extrêmement limité pour une surface aussi riche que f(x, y).

En pratique, avec cette architecture réduite :

- Le réseau capture la tendance globale (zones hautes et basses) mais rate les détails fins.
- Les oscillations rapides de $\sin(\sqrt{x^2+y^2})$ sont mal représentées.
- La loss finale reste significativement plus haute qu'avec l'architecture [2, 64, 64, 1].
- Le phénomène observé s'appelle le sous-ajustement (underfitting) : le modèle n'a pas assez de capacité.

Conclusion — Question 2

Non, l'architecture [2, 4, 1] est insuffisante pour capturer les détails de f(x, y).
L'architecture [2, 64, 64, 1] avec ses ~8 500 paramètres offre la capacité

nécessaire pour approximer fidèlement cette surface complexe.

Question 3 — Optimisation avec Momentum

Comment l'ajout d'un terme de Momentum affecterait-il la convergence ?

Le Momentum est une technique d'optimisation qui accumule une « vélocité » dans les directions du gradient, de façon analogue à une bille qui roule et prend de la vitesse dans les creux :

```
# SGD avec Momentum (coefficient β = 0.9 typiquement)
v[i] = β * v[i] + (1 - β) * dW[i]    # vélocité accumulée
W[i] = W[i] - lr * v[i]                # mise à jour avec momentum
```

Les bénéfices concrets du Momentum par rapport au SGD pur :

- Convergence plus rapide : le gradient accumulé s'amplifie dans les directions constantes.
- Moins sensible aux minima locaux : la vélocité permet de « franchir » de petits plateaux.
- Réduction du bruit : les oscillations dues aux mini-batchs sont amorties.
- Moins de sensibilité au learning rate : on peut souvent utiliser un lr plus grand.

En pratique sur notre problème, l'ajout du Momentum (β = 0.9) permettrait de réduire le nombre d'époques nécessaires pour atteindre la même loss finale, passant typiquement de 1000 à environ 400-600 époques. C'est la base de l'optimiseur Adam (Adaptive Moment Estimation), qui combine Momentum et adaptation du learning rate.

Conclusion — Question 3

Le Momentum accélère significativement la convergence en accumulant la vélocité dans les directions persistantes du gradient. Il réduit les oscillations et permet d'atteindre une meilleure loss en moins d'époques. C'est la base d'Adam et RMSProp.

Conclusion

Ce TP a permis d'implémenter de zéro un Perceptron Multicouche complet avec NumPy, en respectant toutes les contraintes imposées : vectorisation, backpropagation manuelle, et absence de frameworks. Les principaux résultats obtenus sont :

- Le MLP [2, 64, 64, 1] apprend avec succès à approximer la surface f(x, y) non-linéaire.
- La loss MSE converge de ~1.17 à ~0.024 en 1000 époques.
- La comparaison visuelle montre une bonne fidélité entre la prédiction et la fonction originale.
- Les questions de réflexion soulèvent des points fondamentaux : rôle des non-linéarités, capacité du réseau, et accélération de l'optimisation avec le Momentum.

Ce TP constitue une base solide pour comprendre le fonctionnement interne des réseaux de neurones, avant d'aborder des frameworks plus avancés comme PyTorch ou TensorFlow.